

The Waterfall Model

A Classical Software Development Life Cycle Approach

Course	Software Engineering
Topic	Software Development Life Cycle (SDLC)
Model	Waterfall
Document Type	Research Report

An in-depth study of the Waterfall model: its phases, principles, strengths, limitations, and place in modern software engineering.

Table of Contents

1.	Abstract	3
2.	Introduction to the SDLC	3
3.	Origin and Historical Background	4
4.	The Phases of the Waterfall Model	5
5.	Diagrammatic Representation	7
6.	Core Principles and Characteristics	8
7.	Advantages of the Waterfall Model	9
8.	Disadvantages and Limitations	10
9.	Variants of the Waterfall Model	11
10.	When to Use the Waterfall Model	12
11.	Waterfall vs. Agile: A Comparative Analysis	13
12.	Real-World Applications	14
13.	Conclusion	15
14.	References	15
A.	Appendix: Anticipated Examination Questions	16

1. Abstract

The Waterfall model is one of the earliest and most influential software development life cycle (SDLC) methodologies. First formally described by Dr. Winston W. Royce in 1970, the model structures the software development process as a strictly sequential flow of phases: requirements, design, implementation, testing, deployment, and maintenance. Each phase must be completed in full before the next begins, and each produces formal documentation that serves as input to the subsequent phase. This research paper presents a comprehensive study of the Waterfall model, covering its historical origin, phases, core principles, advantages, limitations, variants, appropriate application contexts, and comparison with modern iterative methodologies such as Agile. The goal is to equip the reader with both a theoretical and practical understanding of when and why the Waterfall model remains relevant in contemporary software engineering.

2. Introduction to the SDLC

The Software Development Life Cycle (SDLC) is the overarching framework that defines the stages through which software moves from an initial idea to a deployed, maintained product. Every SDLC model aims to answer the same fundamental questions: *what* should be built, *how* it should be built, *who* is responsible for each activity, and *when* each activity happens. Different models answer these questions differently. Some favor rigid sequencing and heavy documentation; others favor flexibility and continuous customer feedback.

The Waterfall model sits at one extreme of this spectrum. It assumes that requirements can be fully understood at the start of the project, that design can be completed before coding begins, and that each phase hands off a signed-off document to the next. It is the canonical example of a **plan-driven** approach to software engineering and, for decades, was the default methodology taught in universities and used in industry. Even today, where iterative methods dominate, Waterfall remains an important reference point — both as a historical foundation and as a methodology that still fits certain classes of projects.

3. Origin and Historical Background

The Waterfall model is most often attributed to Dr. Winston W. Royce, who described it in his 1970 paper *"Managing the Development of Large Software Systems."* Interestingly, Royce presented the pure sequential model as an example of a flawed approach and immediately advocated for iteration and prototyping on top of it. However, later practitioners and government contractors adopted the linear version of his diagram as a prescriptive standard, and it is this simplified version that became known as the "Waterfall model."

"I believe in this concept, but the implementation described above is risky and invites failure."

— Winston W. Royce, 1970 (on the pure linear flow)

The model rose to prominence throughout the 1970s and 1980s, becoming the dominant methodology in aerospace, defense, and large-scale government projects. The U.S. Department of

Defense formally endorsed it through standards such as DOD-STD-2167A, which mandated a Waterfall-like process for software contracted to the department. Its decline began in the 1990s with the publication of iterative methods, the Unified Process, and eventually the Agile Manifesto in 2001.

4. The Phases of the Waterfall Model

The Waterfall model is typically described in five to seven sequential phases. The variant presented below uses the most widely accepted six-phase formulation.

4.1 Requirements Analysis

All functional and non-functional requirements are collected from stakeholders and documented in a Software Requirements Specification (SRS). No design or development work begins until this document is reviewed, signed off, and considered complete. The SRS becomes the contract between the development team and the customer.

Primary output: Software Requirements Specification (SRS)

4.2 System and Software Design

The requirements are translated into a blueprint for the software. Architects produce high-level design documents (system architecture, module decomposition, data models, interface definitions) and low-level design documents (class diagrams, algorithms, database schemas). Design decisions are frozen before any code is written.

Primary output: System Design Document (SDD), UML diagrams

4.3 Implementation (Coding)

Developers implement the design in source code. Work is typically divided into modules or units, each coded independently based on the design specification. The goal of this phase is to produce working units of software that conform exactly to the design.

Primary output: Source code organized into modules

4.4 Integration and Testing

Individual modules are integrated into a complete system, and the system is tested against the requirements defined in the SRS. Testing includes unit testing, integration testing, system testing, and acceptance testing. Defects discovered here must be repaired before release.

Primary output: Tested executable system, test reports

4.5 Deployment

The validated system is delivered to the customer and installed in the production environment. User training, data migration, and rollout planning occur in this phase. In classical Waterfall, deployment marks the first time the customer sees the working product.

Primary output: Deployed software, user documentation

4.6 Maintenance

Post-deployment, the software enters a long-lived maintenance phase. Bug fixes, performance tuning, and adaptations to changing environments are performed. Major new features are typically

treated as a new project, not as part of the original Waterfall cycle.

Primary output: Patches, updates, maintenance reports

5. Diagrammatic Representation

The following diagram illustrates the classical Waterfall flow. Each phase cascades into the next, with the output of one phase forming the input of the next. There is no built-in mechanism to return upstream.

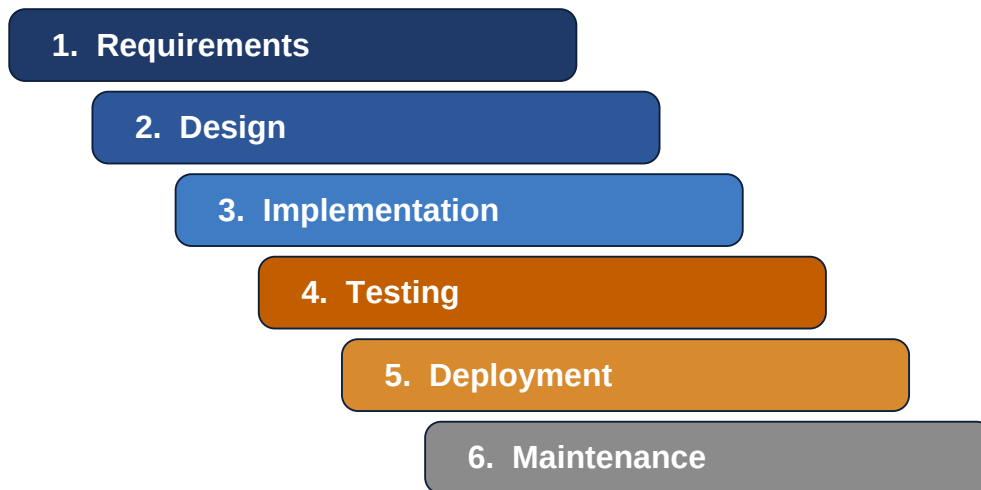


Figure 1 · The classical Waterfall model cascades each phase into the next. Movement is strictly downward.

6. Core Principles and Characteristics

- **Strict sequencing.** Phases must be completed in order. A phase begins only when the previous one is fully complete and documented.
- **Document-driven hand-offs.** Each phase produces formal documents that serve as the authoritative input to the next phase and as the formal contract between stakeholders.
- **Requirements frozen upfront.** All functional and non-functional requirements are expected to be gathered, analysed, and approved before any design or coding begins.
- **No backward flow.** In the pure model, one cannot return to an earlier phase. Changes discovered late are expensive because they force a restart.
- **Clear milestones and deliverables.** Each phase ends with a well-defined milestone and deliverable, making the model highly trackable and auditable.
- **Testing concentrated near the end.** Formal testing begins only after implementation is complete, which means defects are often discovered late in the cycle.
- **Customer involvement at the extremes.** The customer is heavily involved at the beginning (requirements) and the end (acceptance) but rarely in between.

7. Advantages of the Waterfall Model

Simplicity and clarity. The model is easy to understand, teach, and manage, especially for teams new to software engineering.

Well-defined structure. Clear phases and deliverables allow managers to track progress, estimate effort, and allocate resources with high confidence.

Strong documentation. The heavy emphasis on documents makes the project auditable, transferable between teams, and easy to onboard new members into.

Suited to fixed-scope projects. When requirements are stable and fully known, Waterfall offers a predictable path to delivery.

Effective for contractual work. The phase-by-phase sign-off model aligns neatly with fixed-price contracts and regulatory environments.

Quality gates. Each phase transition acts as a formal quality gate, forcing inspection and sign-off before work continues.

8. Disadvantages and Limitations

Inflexibility to change. Real-world requirements rarely stay frozen. Any change discovered after the requirements phase is difficult and costly to accommodate.

Late customer feedback. Because working software is only visible at the end, customers may reject the final product or request fundamental changes that cannot be cheaply accommodated.

Late defect discovery. Bugs rooted in design errors are not found until the testing phase, where they are dramatically more expensive to fix than if caught earlier.

High risk for large projects. The longer the single pass, the greater the risk that assumptions made at the beginning will no longer hold when the project finishes.

Poor fit for research-heavy projects. Projects where the desired behaviour is not yet fully understood — such as new products, startups, or research — cannot freeze requirements and therefore do not fit Waterfall.

Documentation overhead. Producing and maintaining the required documents consumes significant time and effort that could otherwise be spent building software.

No working software until late. Stakeholders have nothing tangible to evaluate during most of the project, which erodes trust and delays feedback.

9. Variants of the Waterfall Model

Several refinements of the pure Waterfall model attempt to address its rigidity while preserving its phase-based structure:

Iterative Waterfall. Permits limited backward flow to the previous phase only. When an issue is found in a phase, the team may return to the one immediately before it to correct errors, then resume forward progress.

V-Model (Verification and Validation). Folds the Waterfall into a V-shape, pairing each development phase with a corresponding testing phase: Requirements ↔ Acceptance Testing, Design ↔ Integration Testing, Implementation ↔ Unit Testing. Emphasises that testing activities should be planned in parallel with development.

Sashimi Waterfall. Allows overlapping of adjacent phases so that later phases may begin before earlier phases are fully complete. Named after the way slices of sashimi overlap on a plate.

Incremental Waterfall. Divides the system into smaller increments, each delivered through a mini-Waterfall cycle. Offers early partial deliveries while preserving the sequential structure within each increment.

10. When to Use the Waterfall Model

Waterfall is an appropriate choice when the following conditions hold:

- Requirements are clearly defined, understood, and unlikely to change.
- The technology stack and tools are well-established and stable.
- The project is short enough that the linear pass completes before requirements drift.
- The customer expects a formal, document-heavy process (typical of government, defence, aerospace, and regulated industries).
- The team is distributed or new, and requires rigid structure to coordinate work.
- There is a fixed-price contract that forbids mid-project scope changes.

Conversely, Waterfall is a poor choice when requirements are fuzzy or evolving, when stakeholder feedback is critical during development, when the team is building a new kind of product, or when time-to-market is more important than documentation.

11. Waterfall vs. Agile: A Comparative Analysis

The Agile family of methodologies (Scrum, Kanban, XP) emerged explicitly as a response to the limitations of plan-driven approaches like Waterfall. The table below summarises the core contrasts.

Aspect	Waterfall	Agile (Scrum)
Approach	Linear, sequential, single pass	Iterative, incremental, many short cycles
Requirements	Frozen upfront in the SRS	Evolve continuously each sprint
Customer feedback	End of project	Every sprint (1–4 weeks)
Documentation	Heavy, formal, upfront	Light, 'just enough'
Handles change	Poorly; changes are expensive	Embraces change as a first-class concept
Delivery	Single big-bang release	Continuous incremental releases
Risk discovery	Late (during testing phase)	Early (each sprint review)
Team structure	Phase-specialised roles	Cross-functional self-organising teams
Best suited for	Fixed, well-understood scope	Fuzzy or evolving requirements
Planning horizon	Entire project planned upfront	Sprint-by-sprint planning
Testing	Concentrated near the end	Continuous, throughout every sprint

Table 1 · Comparison between Waterfall and Agile methodologies.

12. Real-World Applications

Despite the rise of Agile, the Waterfall model is still used — and in some domains, still preferred — today. Notable examples include:

Aerospace and defence systems. Mission-critical systems such as avionics, radar, and flight control software require exhaustive documentation and traceability. These industries often mandate Waterfall-like processes through standards such as DO-178C and MIL-STD-498.

Medical devices. Regulators such as the FDA require formal documentation of requirements, design, verification, and validation. A strict phase-based process simplifies compliance with standards like IEC 62304.

Government and public-sector systems. Government contracts typically require fixed scope, fixed budget, and formal sign-off at each milestone — a natural fit for Waterfall.

Construction and infrastructure software. Systems tightly coupled to physical infrastructure (bridges, power plants, railways) cannot be iterated easily and benefit from the up-front rigour of Waterfall.

Academic and research projects. Short, well-defined projects with clear goals and fixed deadlines — such as university course projects — sometimes map well onto a compressed Waterfall pass.

13. Conclusion

The Waterfall model played a foundational role in establishing software engineering as a disciplined field. Its emphasis on up-front planning, formal documentation, and phase-based sign-off brought structure to an industry that had grown organically and chaotically. Although modern practice has largely shifted towards iterative and Agile approaches — which better handle evolving requirements and rapid feedback — the Waterfall model remains an important reference point. It is still the right choice for projects with fixed scope, stable technology, strict regulatory requirements, or contractual constraints. Understanding both its strengths and its limitations equips software engineers to choose the right process for the right project, which is ultimately the goal of studying software development life cycles.

14. References

- [1] Royce, W. W. (1970). *Managing the Development of Large Software Systems*. Proceedings of IEEE WESCON, 26, 1–9.
- [2] Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson.
- [3] Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill.
- [4] Boehm, B. W. (1988). A Spiral Model of Software Development and Enhancement. *Computer*, 21(5), 61–72.

-
- [5] Beck, K. et al. (2001). *Manifesto for Agile Software Development*. Retrieved from agilemanifesto.org.
- [6] IEEE Std 830-1998. *IEEE Recommended Practice for Software Requirements Specifications*.
- [7] IEC 62304:2006. *Medical device software — Software life cycle processes*.

Appendix A — Anticipated Examination Questions

The following questions are the most likely to be asked by the examiner when defending this research paper. Each answer is written in a form suitable for an oral defence (viva) and has been kept concise so that the student can rehearse the talking points.

Q1. Who introduced the Waterfall model and when?

The model is attributed to Dr. Winston W. Royce, who described it in his 1970 paper *Managing the Development of Large Software Systems*. Ironically, Royce presented the purely linear form as an example of a flawed approach and recommended iteration on top of it, but later practitioners adopted the linear version and it became known as the Waterfall model.

Q2. What are the phases of the Waterfall model?

Requirements analysis, system and software design, implementation (coding), integration and testing, deployment, and maintenance. Each phase must be completed before the next begins.

Q3. Why is it called 'Waterfall'?

Because work flows strictly downward from one phase to the next, like water cascading down a waterfall. Once the work leaves a phase, it cannot flow back upstream in the pure model.

Q4. What are the main advantages of the Waterfall model?

Simplicity, clear milestones, strong documentation, predictable planning, suitability for fixed-scope projects and contractual work, and the presence of quality gates between phases.

Q5. What are the main disadvantages?

Inflexibility to change, late customer feedback, late defect discovery, high risk for large projects, unsuitability for projects with evolving requirements, heavy documentation overhead, and the absence of working software until the end of the project.

Q6. When should the Waterfall model be used?

When requirements are well understood and stable, the technology is mature, the project is short enough to complete before requirements drift, or when regulatory or contractual constraints demand formal, document-based, phase-gated processes — typical in aerospace, defence, medical devices, and government systems.

Q7. How does Waterfall differ from Agile?

Waterfall is linear, plan-driven, document-heavy, and delivers software at the end of the project. Agile is iterative, feedback-driven, lightweight on documentation, and delivers working software every sprint. Waterfall freezes requirements upfront; Agile expects them to evolve.

Q8. What is the V-Model and how does it relate to Waterfall?

The V-Model is a Waterfall variant that folds the model into a V-shape, pairing each development phase with a corresponding testing phase. Requirements pair with acceptance testing, design with integration testing, and implementation with unit testing. It emphasises that testing should be planned alongside development, not postponed to the end.

Q9. Why did Royce criticise the pure Waterfall model?

Royce argued that the pure linear flow was risky because it did not account for feedback, prototyping, or iteration. He proposed adding feedback loops and building a prototype first. His critique was largely ignored, and the simplified linear diagram became the model that carries his name.

Q10. Give a real-world example of a project where Waterfall is appropriate.

Avionics software for a commercial aircraft. The requirements are rigorously specified by aviation authorities, the system must be formally verified against standards such as DO-178C, changes are not tolerated late in the cycle, and exhaustive documentation is a legal necessity. Waterfall (or its V-Model variant) fits this environment naturally.

Q11. What happens if a requirement changes during the coding phase?

In the pure Waterfall model, the team must formally return to the requirements phase, update the SRS, update the design, and re-enter implementation. This is costly and disruptive, which is why pure Waterfall is poorly suited to environments where change is common.

Q12. What role does documentation play in Waterfall?

Documentation is the backbone of the methodology. Each phase produces a formal document (SRS, SDD, test plan, user manual) that serves both as the handoff to the next phase and as the contractual deliverable. Documents make the process auditable, traceable, and transferable between teams.

Q13. Is the Waterfall model still used today?

Yes, in domains that require formal verification, regulatory compliance, or contractual fixed-scope delivery: aerospace, defence, medical devices, government systems, and certain infrastructure projects. It has been largely replaced by Agile in commercial software, startups, and web development.

Q14. What is the difference between the Waterfall model and the Spiral model?

The Spiral model, introduced by Barry Boehm in 1988, combines elements of Waterfall with iterative prototyping and explicit risk analysis. Instead of a single linear pass, the Spiral model cycles through risk-driven iterations. Waterfall is linear and plan-driven; Spiral is iterative and risk-driven.

Q15. What is meant by 'requirements freezing' in the Waterfall model?

Requirements freezing is the principle that once the Software Requirements Specification is signed off, no new requirements may be added and existing ones may not be changed without a formal change-control process. This discipline is what gives Waterfall its predictability — and also what makes it brittle in the face of real-world change.

Q16. What is a 'quality gate' in the Waterfall model?

A quality gate is a formal checkpoint at the end of each phase where the output documents are reviewed, verified, and signed off before the team is allowed to proceed to the next phase. Quality gates enforce the discipline of the model and prevent defects from silently propagating downstream.

Q17. Why is fixing a bug more expensive when discovered late in Waterfall?

Because a late bug may require revisiting decisions made in earlier phases — requirements, design, or code — and re-verifying all downstream artefacts. Barry Boehm's classic research on

software economics showed that defects found in production can cost up to 100× more to fix than defects caught during requirements analysis.

Q18. Can Waterfall and Agile be combined?

Yes. Hybrid approaches (sometimes called 'Water-Scrum-Fall') use Waterfall for the high-level planning and release milestones, while running Agile sprints within the implementation phase. This can be a pragmatic compromise in organisations that need both formal contractual structure and internal iterative flexibility.

Q19. What is the role of the customer in the Waterfall model?

The customer is deeply involved at the start (providing and approving requirements) and at the end (accepting the delivered system). Between these two points, the customer typically has little visibility. This is one of the main criticisms of the model: the customer may only discover that the product does not match their real needs at acceptance time.

Q20. What is the biggest lesson learned from the decline of pure Waterfall?

That software requirements are inherently uncertain, that feedback is more valuable than prediction, and that working software delivered frequently is a better measure of progress than documentation completeness. These lessons directly motivated the Agile Manifesto of 2001.

End of document.